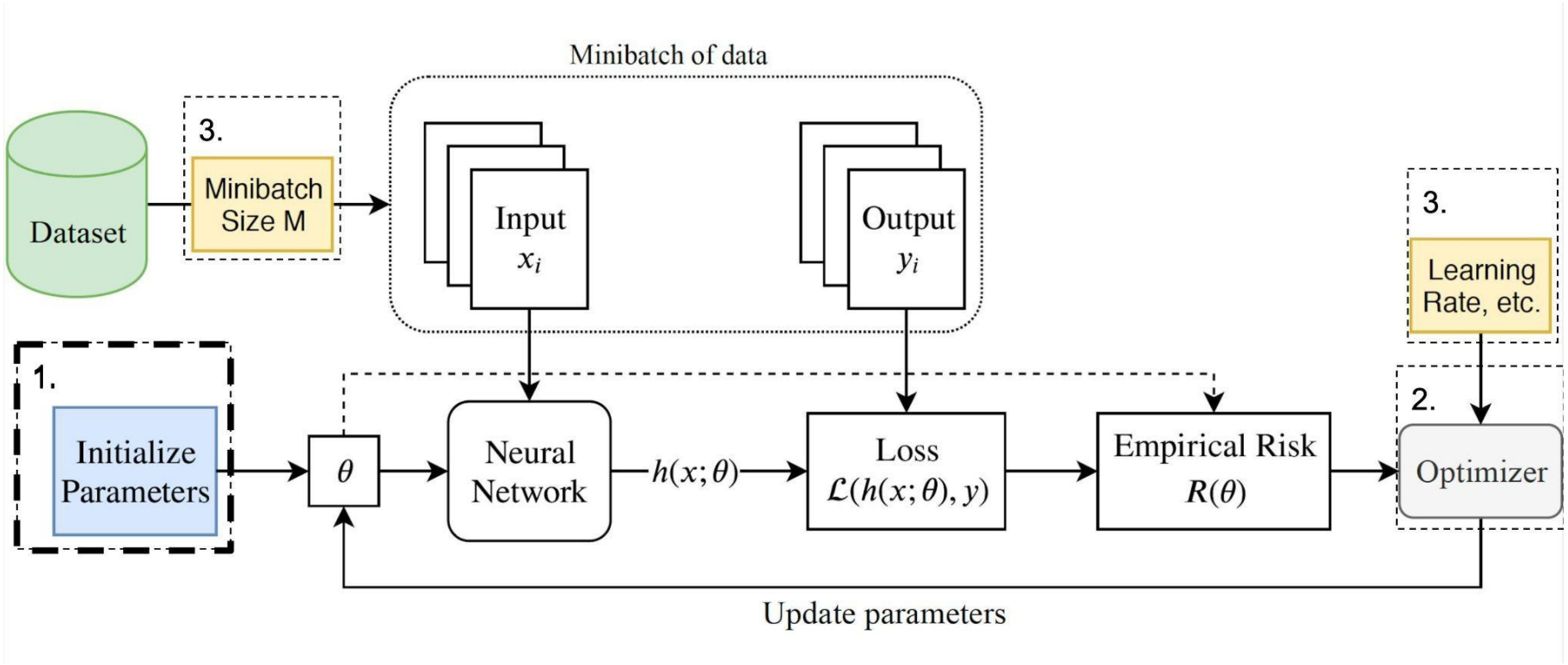
Artificial Intelligence
I4-AMS

DORN Dawin & VANN Visal

Lecture 4: Optimization Algorithms and Hyperparameter Tuning

Institute of Technology of Cambodia · November 2, 2026

The Neural Network Training Loop



1. **Initialize the parameters** (Lecture 3): break the symmetry, He or Xavier scale.
2. **The optimizer** and its learning rate: today.
3. **The mini-batch size** M, and the other hyperparameters: today.

Figure: CSC413 tutorial "How to train your neural network", University of Toronto.

Remember: (Batch) Gradient Descent

Update equation:

$$\theta^{\text{new}} = \theta^{\text{old}} - \alpha \nabla_\theta J(\theta)$$

α = step size or learning rate

i.e., for each layer: $\mathbf{W}^{[l]} := \mathbf{W}^{[l]} - \alpha\, d\mathbf{W}^{[l]}$, $\mathbf{b}^{[l]} := \mathbf{b}^{[l]} - \alpha\, d\mathbf{b}^{[l]}$

J and its gradient use **all** m training examples: one epoch = one step.

Question: what happens when $m = 5{,}000{,}000$?

→ Every single step needs a forward and a backward pass over 5 million examples, and we need thousands of steps. Use a small part of the data per step: **mini-batches**.

2. Mini-batch Gradient Descent

$\mathbf{X}^{\{1\}}$	rows 1 – 1000	$(1000, n_x)$
$\mathbf{X}^{\{2\}}$	rows 1001 – 2000	$(1000, n_x)$
	⋮	
$\mathbf{X}^{\{5000\}}$	last 1000 rows	$(1000, n_x)$

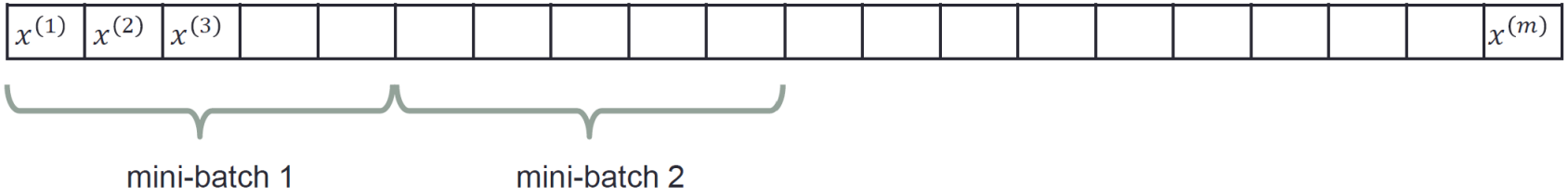
$m = 5{,}000{,}000$ examples. Split the rows of $\mathbf{X}$ and $\mathbf{Y}$ into mini-batches of 1000 examples: 5000 mini-batches $\mathbf{X}^{\{t\}}, \mathbf{Y}^{\{t\}}$.

Notation	Meaning
$\mathbf{x}^{(i)}$	example i (round brackets)
$\mathbf{Z}^{[l]}$	layer l (square brackets)
$\mathbf{X}^{\{t\}}, \mathbf{Y}^{\{t\}}$	mini-batch t (curly brackets)

Batch gradient descent: all m examples per step. **Mini-batch** gradient descent: one mini-batch per step.

2. Mini-batch Gradient Descent

- 3 variants of gradient descent
  - **Batch GD:** update parameters after going through all training examples (1 epoch)
  - **Stochastic GD:** update parameters after each training example
  - **Mini-batch GD:** update parameters after a small batch of training examples (1 epoch might contain multiple mini-batches)



1 epoch --> all sample, Example: 1000 sample -> we device into 10 mini batch --> run 100 iteration

Mini-batch Gradient Descent

for epoch = 1, 2, …:

  for $t = 1, \ldots, 5000$:

    Forward on $\mathbf{X}^{\{t\}}$: $\mathbf{Z}^{[1]} = \mathbf{X}^{\{t\}}\mathbf{W}^{[1]} + \mathbf{b}^{[1]}$, $\mathbf{A}^{[1]} = g^{[1]}(\mathbf{Z}^{[1]})$, … , $\mathbf{A}^{[L]}$

    Cost on this mini-batch: $J^{\{t\}} = \frac{1}{1000}\sum_{i=1}^{1000}\mathcal{L}\left(\hat{y}^{(i)}, y^{(i)}\right) + \frac{\lambda}{2\cdot 1000}\sum_{l}\|\mathbf{W}^{[l]}\|_F^2$

    Backprop on $J^{\{t\}}$: $d\mathbf{W}^{[l]}, d\mathbf{b}^{[l]}$

    $\mathbf{W}^{[l]} := \mathbf{W}^{[l]} - \alpha\, d\mathbf{W}^{[l]}, \quad \mathbf{b}^{[l]} := \mathbf{b}^{[l]} - \alpha\, d\mathbf{b}^{[l]}$

One epoch = one pass through the training set.

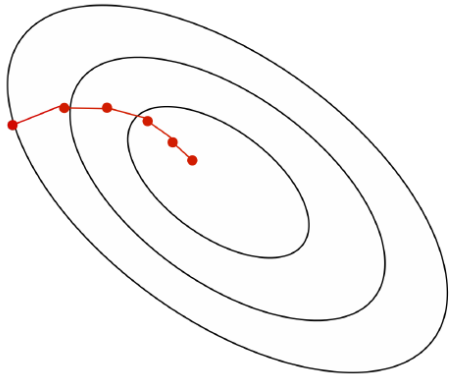
Question: $m = 5{,}000{,}000$ and mini-batches of 1000: how many gradient steps does one epoch take?

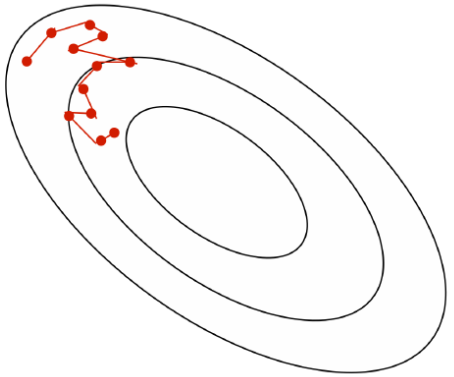
→ Batch gradient descent: **1** step per epoch. Mini-batch gradient descent: **5000** steps per epoch, each as cheap as a pass over 1000 examples.

Batch vs. Stochastic Gradient Descent

Batch GD moves directly downhill.

Stochastic GD (one example per step): a noisy direction, but downhill on average.



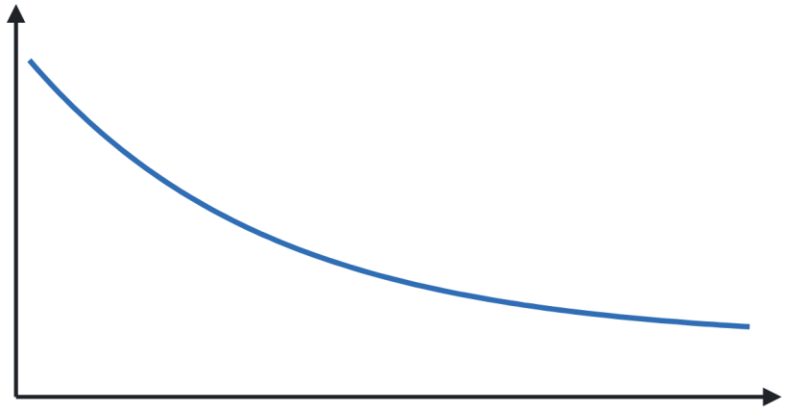


Question: which one has the faster updates? Needs less memory? Has the more stable path?

	Batch GD	Stochastic GD
Speed of one update	slow: all m examples	fast: one example
Memory	the whole dataset	one example
Path	stable	noisy, erratic; can escape a local minimum
Convex J	converges	only if α decays

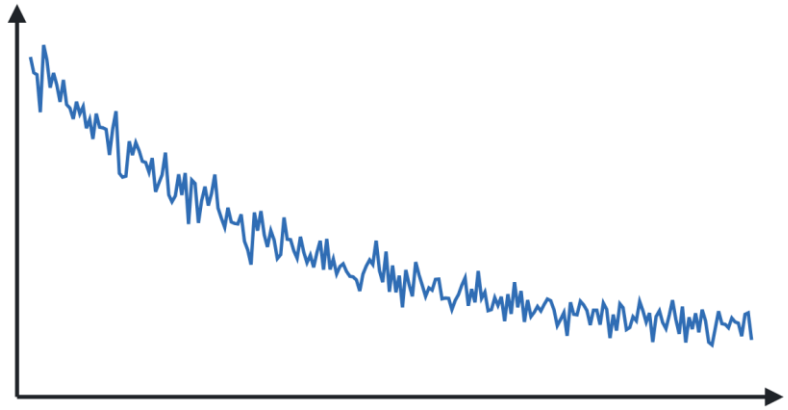
Figure: A. Gao, CSC413 Neural Networks and Deep Learning, University of Toronto.

Training with Mini-batch Gradient Descent



Batch GD: cost vs. iterations

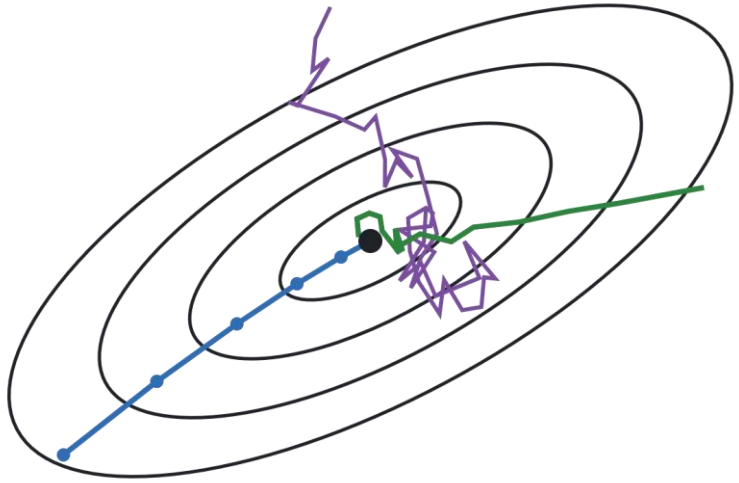
Should decrease at **every** iteration. If it goes up even once: a bug, or α too large.



Mini-batch GD: cost vs. mini-batch t

$J^{\{t\}}$ uses a different mini-batch each time: noisy, but it trends down.

Choosing the Mini-batch Size



Size m: batch GD. Smooth path, but too long per iteration.

Size 1: stochastic GD. Noisy, never settles, loses the speed-up from vectorization.

In between: mini-batch. Fastest: vectorization, and progress before seeing all the data.

- Small training set ($m \leq 2000$): batch gradient descent.
- Otherwise: 64, 128, 256 or 512 (powers of 2 suit the hardware), and one mini-batch must fit in the CPU/GPU memory.

Mini-batches in PyTorch: One Extra Loop

Batch gradient descent

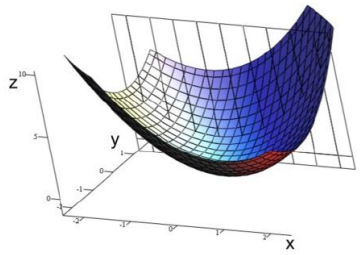
```
for epoch in range(epochs):
    model.train()
    y_logits = model(X_train)     # all m examples
    loss = loss_fn(y_logits, y_train)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()              # 1 step per epoch
```

Mini-batch gradient descent

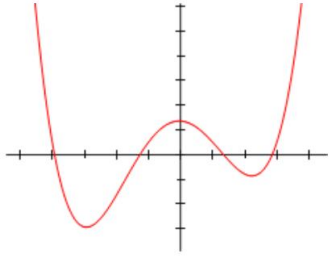
```
loader = DataLoader(TensorDataset(X_train, y_train),
                    batch_size=64, shuffle=True)
for epoch in range(epochs):
    model.train()
    for X_t, y_t in loader:       # mini-batch t
        y_logits = model(X_t)
        loss = loss_fn(y_logits, y_t)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()          # m/64 steps per epoch
```

- shuffle=True: a new random split into mini-batches at every epoch.
- The last mini-batch is smaller when m is not a multiple of 64 (drop_last=True removes it).

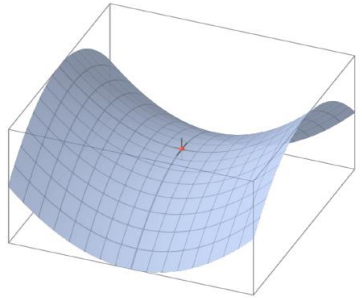
3. The Optimization Landscape



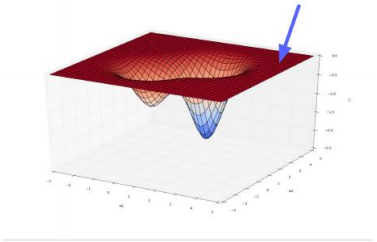
convex functions



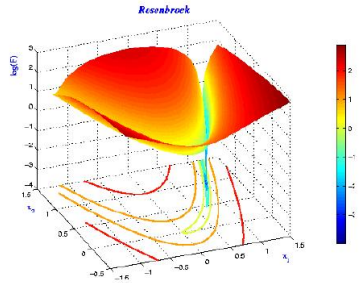
local minima



saddle point



plateaux

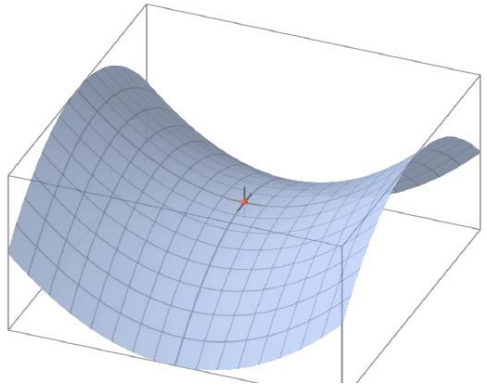


narrow ravine

- **Convex** (linear and logistic regression): bowl-shaped, a single minimum.
- **Neural networks are not convex.** Swap two hidden units, with their weights: a different θ, the same cost. So every minimum has many copies.
- Today: the features that make gradient descent **slow**, and how to fix them.

Figure: A. Gao, CSC413 Neural Networks and Deep Learning, University of Toronto.

Local Minima and Saddle Points





At a saddle point the gradient is 0, but it is a minimum in one direction and a maximum in another.

With 20,000 parameters, a minimum must curve **up** in all 20,000 directions: most zero-gradient points are saddles.

Saddle points

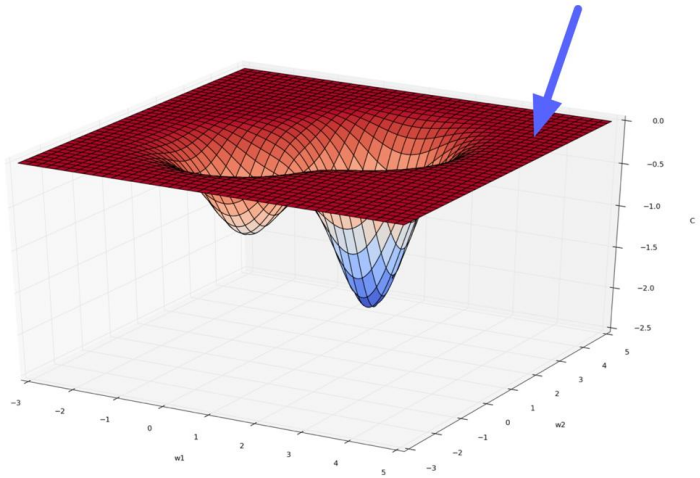
Exactly on one, gradient descent is stuck. Example: all weights 0, so every hidden unit stays identical (Lecture 3). **Fix:** random initialization breaks the symmetry; momentum and Adam move away faster.

Local minima

Hard to diagnose, but in neural networks the local minima found by training are usually **good enough**. Focus on converging fast. If in doubt: random restarts, keep the best run.

Figure: A. Gao, CSC413 Neural Networks and Deep Learning, University of Toronto.

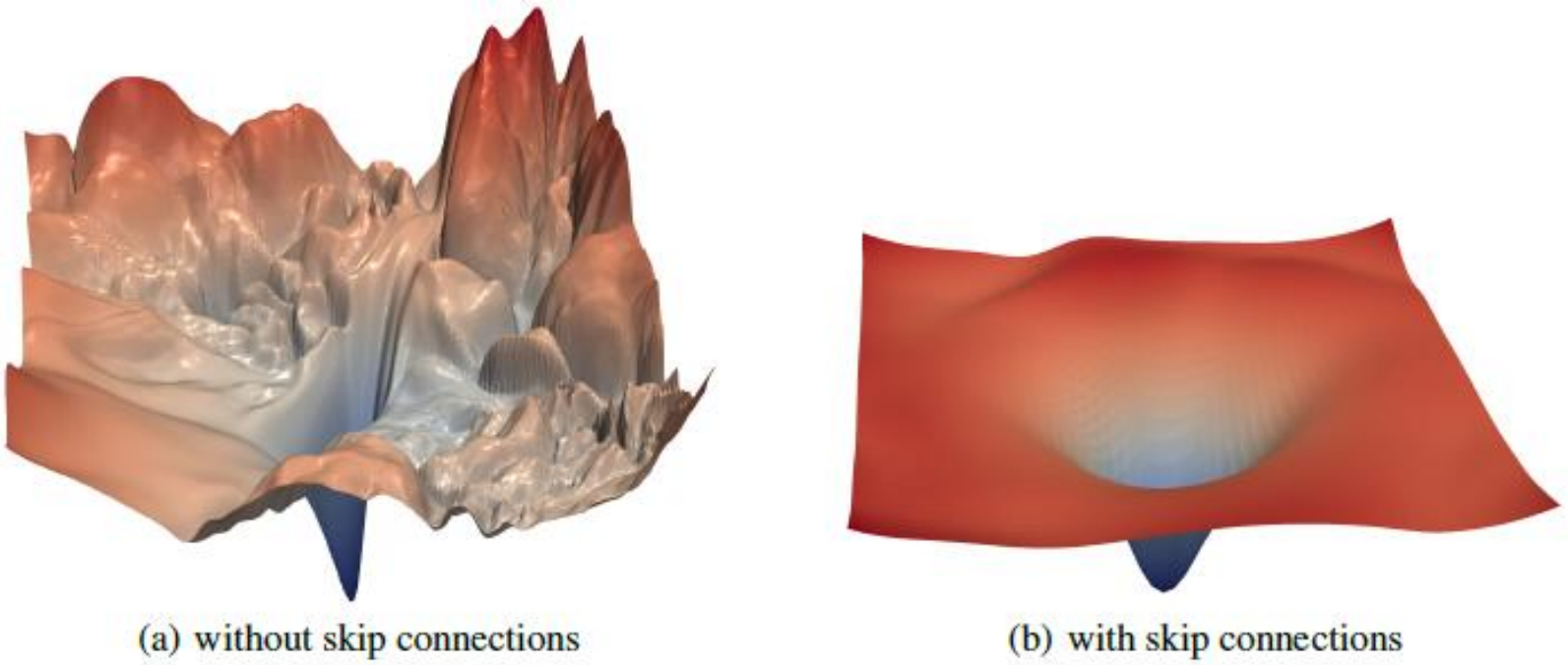
Plateaus: Flat Regions



- A flat region (blue arrow): the gradient is almost 0, so the parameters barely move.
- **Causes:** saturated units (sigmoid or tanh near its extremes), dead ReLUs (input always negative), hard thresholds or 0-1 loss.
- **Diagnose:** plot histograms of the activations.
- **Fix:** the right initial scale (Xavier or He), ReLU in hidden layers, a small positive bias (e.g. 0.1) for ReLU units; momentum and Adam cross a plateau faster.

Figure: A. Gao, CSC413 Neural Networks and Deep Learning, University of Toronto.

What Real Loss Surfaces Look Like



(a) without skip connections

(b) with skip connections

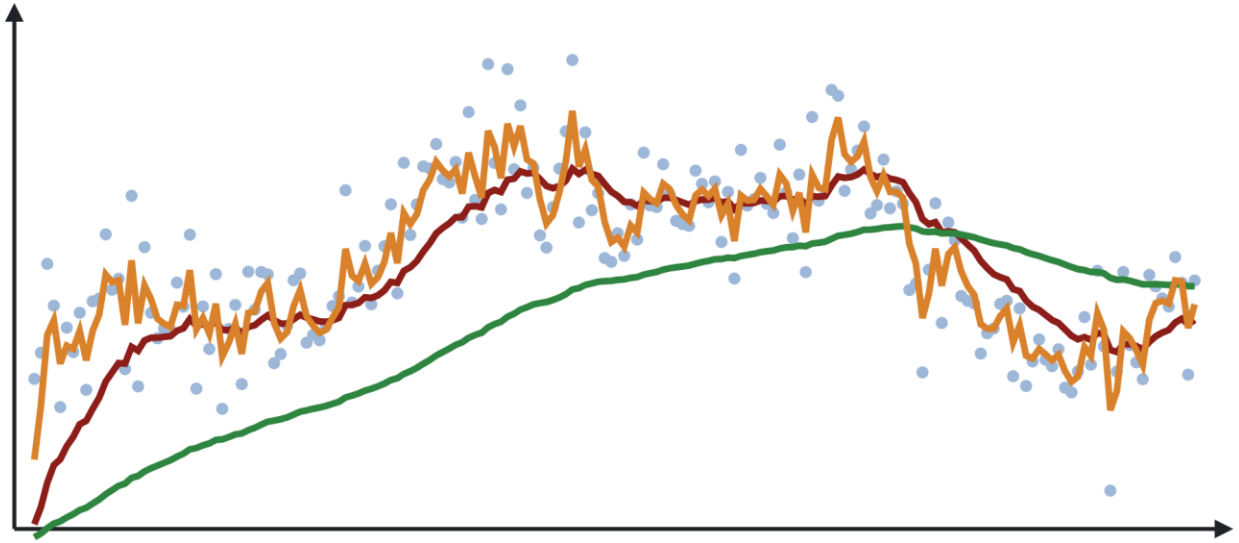
Question: which surface is easier for gradient descent? Why?

→ (b): smooth and nearly convex around the minimum. On (a), a step that looks good locally can lead anywhere. The **architecture** shapes the landscape: skip connections and batch norm make it smoother.

Figure: H. Li et al., Visualizing the loss landscape of neural nets, NeurIPS 2018 (ResNet-56 on CIFAR-10).

4. Exponentially Weighted Averages

Daily temperature θ_t over half a year (blue dots), and its moving averages:



$$v_t = \beta\, v_{t-1} + (1-\beta)\,\theta_t$$

v_t is roughly an average over the last $\frac{1}{1-\beta}$ values:

- $\beta = 0.9$: about 10 days
- $\beta = 0.98$: about 50 days, smoother but shifted right (adapts slowly)
- $\beta = 0.5$: about 2 days, noisy

Understanding Exponentially Weighted Averages

$$
\begin{aligned}
v_{100} &= 0.1\,\theta_{100} + 0.9\,v_{99} \\
&= 0.1\,\theta_{100} + 0.1 \cdot 0.9\,\theta_{99} + 0.1 \cdot 0.9^2\,\theta_{98} + 0.1 \cdot 0.9^3\,\theta_{97} + \cdots
\end{aligned}
$$

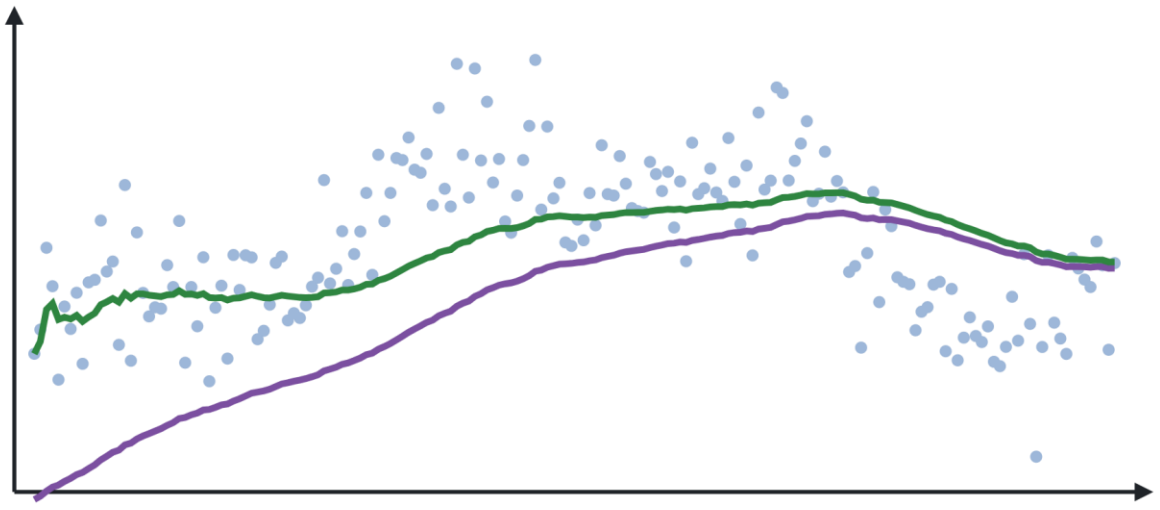
The weights decrease exponentially. After $\frac{1}{1-\beta}$ steps they have dropped to about $1/e$ of the first one:

$$0.9^{10} \approx 0.35 \approx \frac{1}{e} \qquad 0.98^{50} \approx 0.36 \approx \frac{1}{e} \qquad (1-\varepsilon)^{1/\varepsilon} \approx \frac{1}{e}$$

```
v = 0
for theta_t in data:
    v = beta * v + (1 - beta) * theta_t
```

One variable and one line of code per quantity: far cheaper than storing the last 50 values and averaging them.

Bias Correction



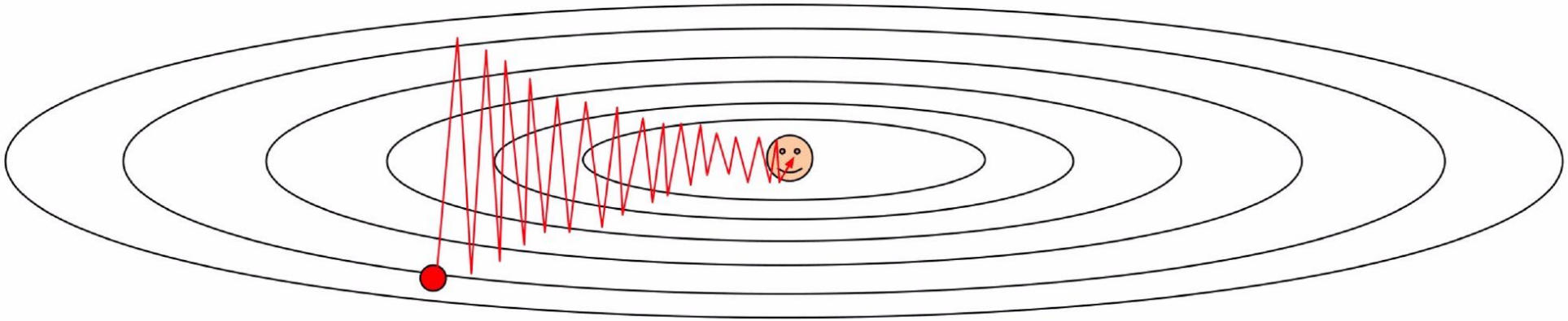
$\beta = 0.98$, $v_0 = 0$: starts far too low

with bias correction

$$v_t^{\text{corrected}} = \frac{v_t}{1 - \beta^t}$$

As t grows, $\beta^t \to 0$ and the correction disappears: it matters only during the first steps.

5. Gradient Descent with Momentum



SGD in a ravine: slow progress along the flat direction, wild oscillations along the steep one. Now imagine a **ball rolling down the hill**: it follows the slope at first, then its momentum keeps it moving.

In the steep direction, momentum dampens oscillations by canceling gradients in opposite directions. In the flat direction, it builds up velocity by accumulating small gradients in the same direction.

→ It **dampens** the oscillations by **cancelling** gradients of opposite signs, and **builds up** velocity by **accumulating** gradients of the same sign.

Figure: A. Gao, CSC413 Neural Networks and Deep Learning, University of Toronto.

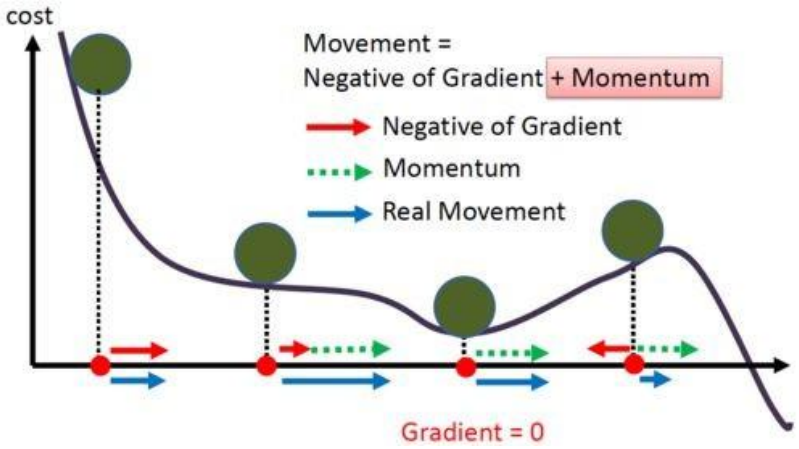
Momentum: Implementation

Initialize $\mathbf{V}_{d\mathbf{W}} = \mathbf{0}, \mathbf{V}_{d\mathbf{b}} = \mathbf{0}$ (same shapes as $\mathbf{W}, \mathbf{b}$)

On iteration t: compute $d\mathbf{W}, d\mathbf{b}$ on the current mini-batch

$$\mathbf{V}_{d\mathbf{W}} = \beta\,\mathbf{V}_{d\mathbf{W}} + (1-\beta)\,d\mathbf{W}, \quad \mathbf{V}_{d\mathbf{b}} = \beta\,\mathbf{V}_{d\mathbf{b}} + (1-\beta)\,d\mathbf{b}$$

$$\mathbf{W} := \mathbf{W} - \alpha\,\mathbf{V}_{d\mathbf{W}}, \quad \mathbf{b} := \mathbf{b} - \alpha\,\mathbf{V}_{d\mathbf{b}}$$



Momentum carries the ball through a point where the gradient is 0.

Hyperparameters

α and β. $\beta = 0.9$ (about 10 gradients) works well almost always. $\beta = 0$: plain gradient descent.

- "Accelerate" learning by accumulating some "velocity/momentum" using the past gradients

Figure: CSC413 tutorial "How to train your neural network", University of Toronto.

RMSprop

Root mean square prop: Adagrad with a leaky sum. Divide each gradient by a running average of its own size.

On iteration t: compute $d\mathbf{W}, d\mathbf{b}$ on the current mini-batch

$$\mathbf{S}_{d\mathbf{W}} = \beta_2\, \mathbf{S}_{d\mathbf{W}} + (1-\beta_2)\, d\mathbf{W}^2 \qquad \text{(elementwise square)}$$

$$\mathbf{S}_{d\mathbf{b}} = \beta_2\, \mathbf{S}_{d\mathbf{b}} + (1-\beta_2)\, d\mathbf{b}^2$$

$$\mathbf{W} := \mathbf{W} - \alpha\, \frac{d\mathbf{W}}{\sqrt{\mathbf{S}_{d\mathbf{W}}} + \varepsilon}, \qquad \mathbf{b} := \mathbf{b} - \alpha\, \frac{d\mathbf{b}}{\sqrt{\mathbf{S}_{d\mathbf{b}}} + \varepsilon}$$

Why it works

Remember how large the gradients have been. Consistently **large** → smaller steps, no overshooting in steep directions. Consistently **small** → larger steps, faster in shallow directions.

Details

$\varepsilon = 10^{-8}$ avoids dividing by zero. Proposed by Hinton in a Coursera course, not in a paper.

Adam: Momentum + RMSprop

$\mathbf{V}_{d\mathbf{W}} = \mathbf{0}, \mathbf{S}_{d\mathbf{W}} = \mathbf{0}, \mathbf{V}_{d\mathbf{b}} = \mathbf{0}, \mathbf{S}_{d\mathbf{b}} = \mathbf{0}$

On iteration t: compute $d\mathbf{W}, d\mathbf{b}$ on the current mini-batch

$$\mathbf{V}_{d\mathbf{W}} = \beta_1 \mathbf{V}_{d\mathbf{W}} + (1-\beta_1)\, d\mathbf{W} \qquad \leftarrow \text{momentum, } \beta_1$$

$$\mathbf{S}_{d\mathbf{W}} = \beta_2 \mathbf{S}_{d\mathbf{W}} + (1-\beta_2)\, d\mathbf{W}^2 \qquad \leftarrow \text{RMSprop, } \beta_2$$

$$\mathbf{V}_{d\mathbf{W}}^{\text{corr}} = \mathbf{V}_{d\mathbf{W}}/(1-\beta_1^t), \quad \mathbf{S}_{d\mathbf{W}}^{\text{corr}} = \mathbf{S}_{d\mathbf{W}}/(1-\beta_2^t) \qquad \leftarrow \text{bias correction}$$

$$\mathbf{W} := \mathbf{W} - \alpha \frac{\mathbf{V}_{d\mathbf{W}}^{\text{corr}}}{\sqrt{\mathbf{S}_{d\mathbf{W}}^{\text{corr}}} + \varepsilon}$$

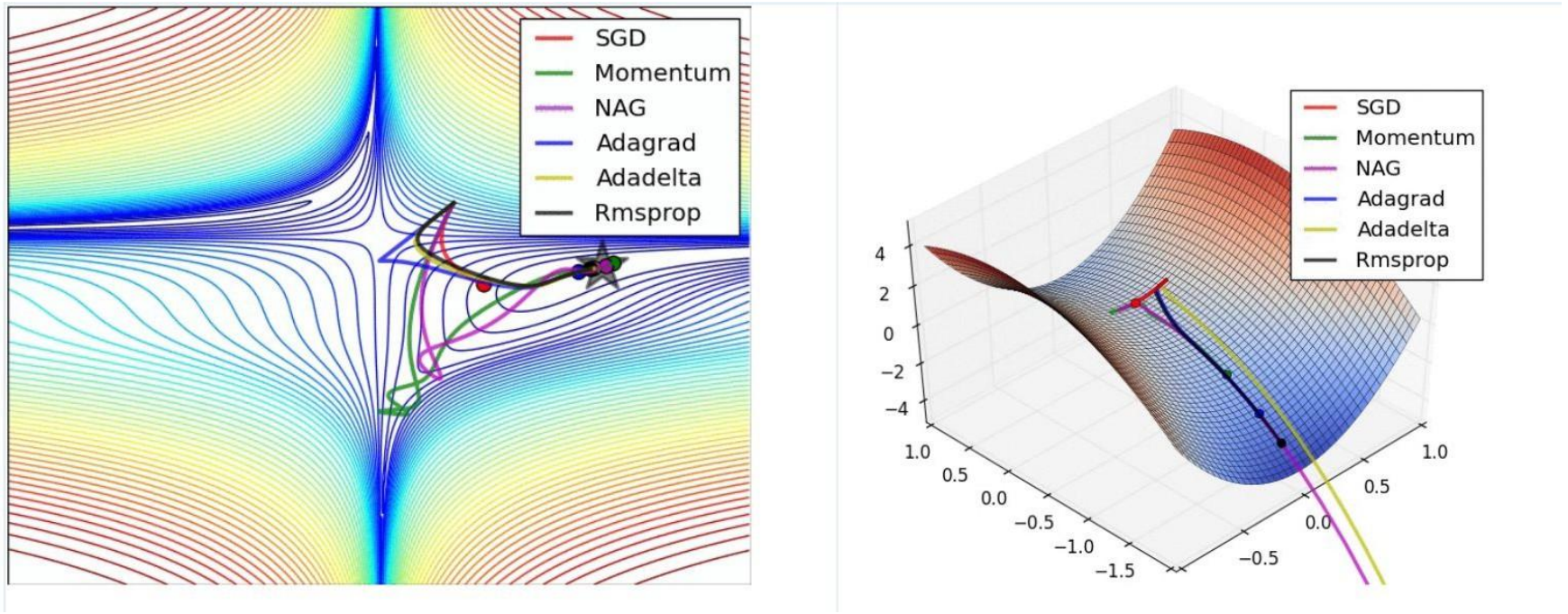
The same four lines for $\mathbf{b}$



Adam = **ada**ptive **m**oment estimation (Kingma and Ba, 2015): $\mathbf{V}$ estimates the mean of the gradient (first moment), $\mathbf{S}$ the mean of its square (second moment).

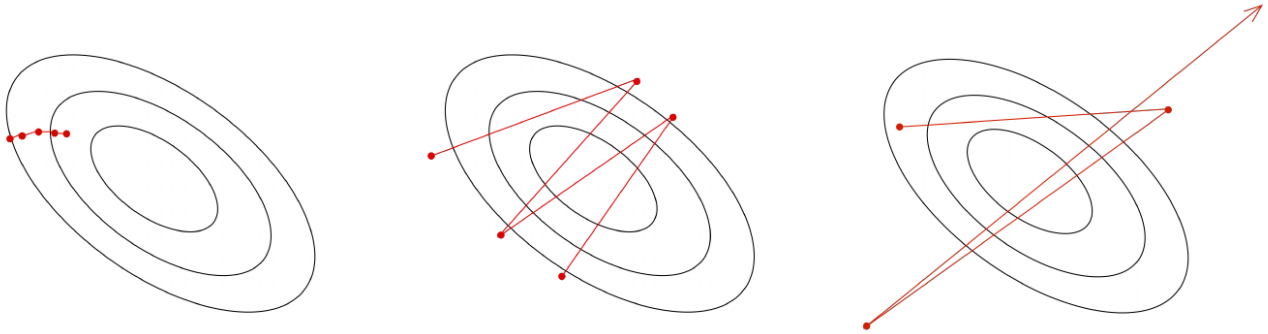
TITLE	CITED BY	YEAR
Adam: A method for stochastic optimization D Kingma, J Ba International Conference on Learning Representations	268604	2015

Optimizers in Action



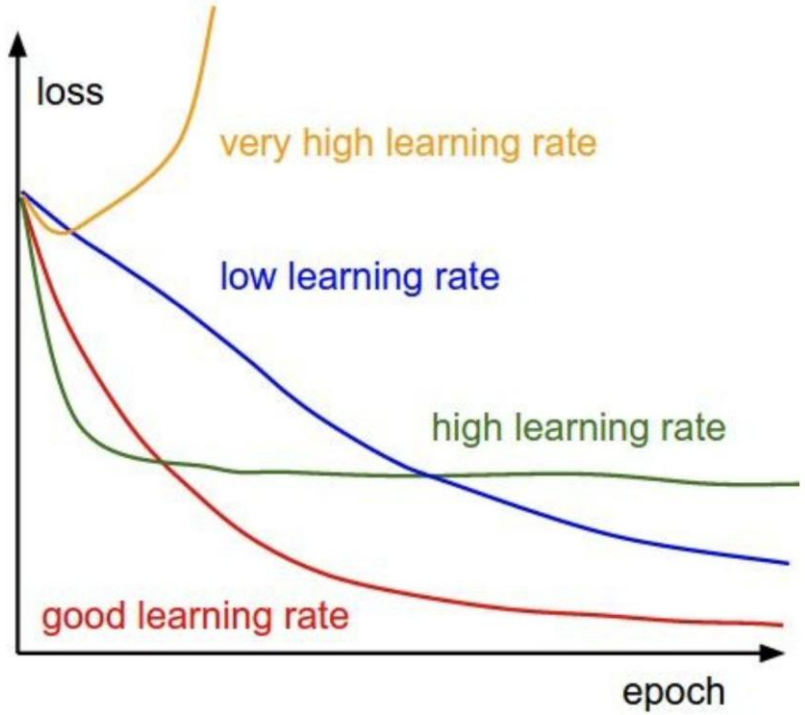
- **Left, a long valley:** momentum and NAG (Nesterov) are led off track, like a rolling ball; NAG corrects its course sooner. Adagrad, Adadelta and RMSprop head in the right direction almost at once.
- **Right, a saddle point:** SGD stays near the saddle; momentum and NAG escape late; the adaptive methods head down the slope quickly.

6. Choosing the Learning Rate



slow progress | oscillations | instability

- Error increases or oscillates wildly → **reduce** α.
- Error decreases consistently but slowly → **increase** α.
- Error stops decreasing, or near the end of training → **reduce** α to remove the fluctuations.

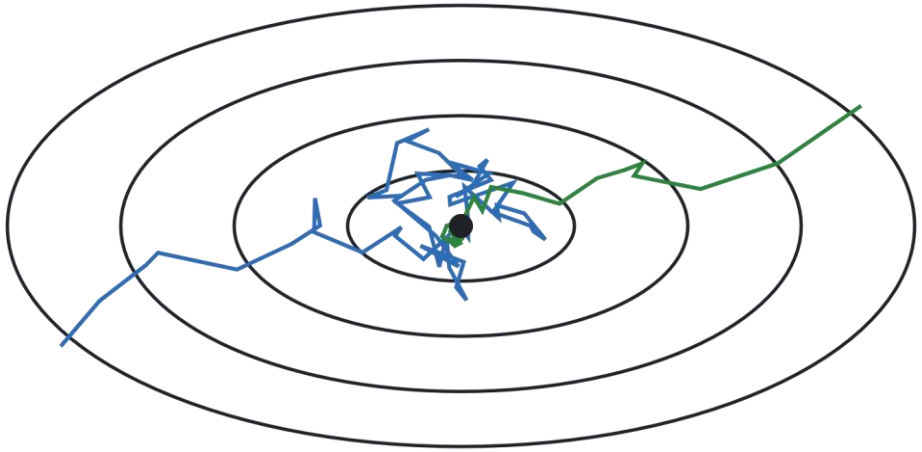


An idealized picture of the loss for different learning rates.

Figures: A. Gao, CSC413, University of Toronto (left); A. Karpathy, CS231n (right). Tips after G. Hinton.

Learning Rate Decay

With mini-batches the steps are noisy. Large steps early to learn fast; smaller steps later to settle near the minimum.



fixed α: keeps wandering around the minimum

decaying α: settles in a tight region

$$\alpha = \frac{1}{1 + \text{decay_rate} \cdot \text{epoch_num}} \alpha_0$$

Example: $\alpha_0 = 0.2$, decay_rate = 1.

Epoch	α
1	0.1
2	0.067
3	0.05
4	0.04

Other Learning Rate Schedules

Schedule	Formula	PyTorch
Exponential decay	$\alpha = 0.95^{\text{epoch}}\,\alpha_0$	ExponentialLR(opt, gamma=0.95)
Inverse square root	$\alpha = \frac{k}{\sqrt{\text{epoch}}}\,\alpha_0$	LambdaLR(opt, f)
Staircase	halve α every N epochs	StepLR(opt, step_size=N, gamma=0.5)
Cosine	slowly, then fast, then slowly	CosineAnnealingLR(opt, T_max)
Manual decay	watch training, reduce by hand	—

```
sched = torch.optim.lr_scheduler.StepLR(opt, step_size=10, gamma=0.5)
for epoch in range(num_epochs):
    train_one_epoch(model, loader, opt)
    sched.step()                        # update alpha once per epoch
```

Learning rate decay is low on the tuning list: first find a good fixed α, then try a schedule.

7. Hyperparameter Tuning: What Matters Most?

Most important

- learning rate α

Next

- momentum $\beta \approx 0.9$
- mini-batch size
- hidden units $n^{[l]}$

Then

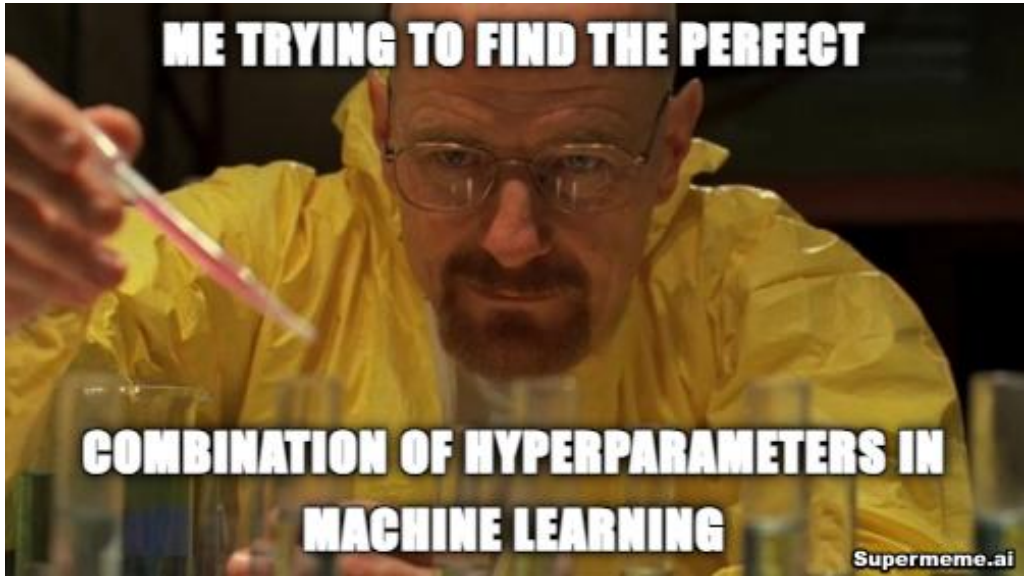
- number of layers L
- learning rate decay

Rarely tuned

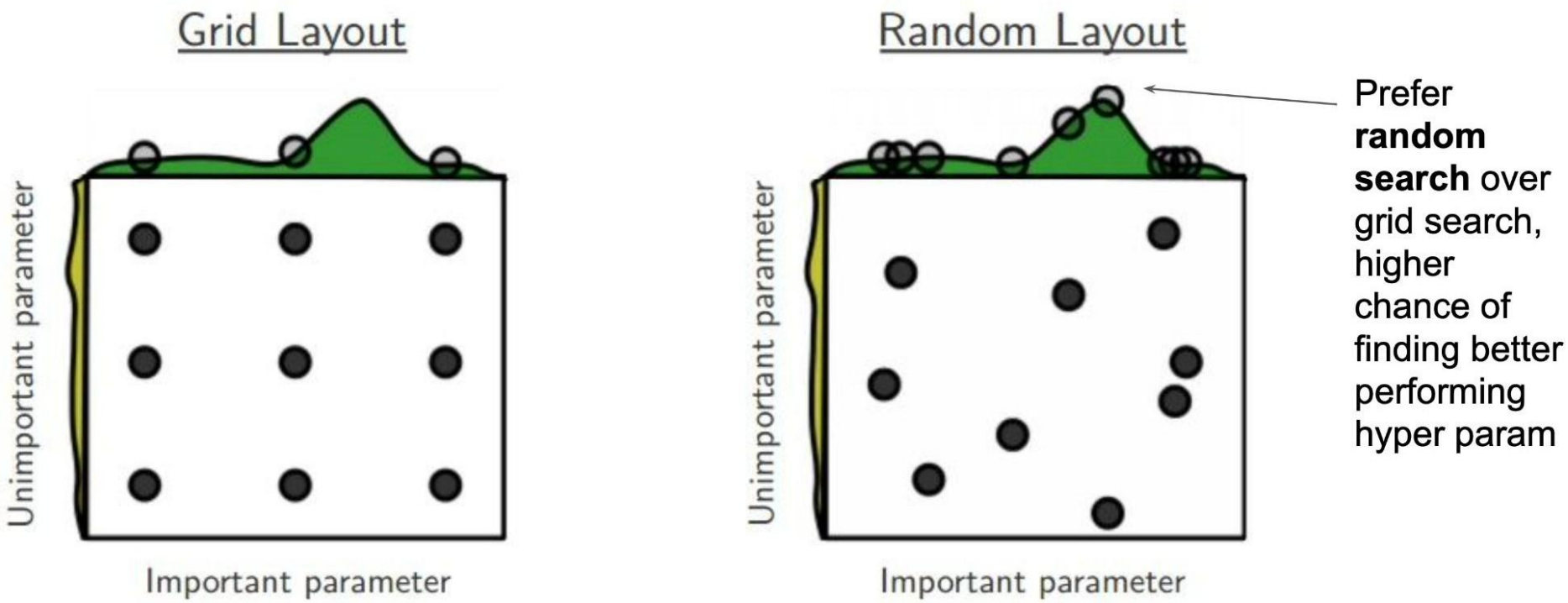
- Adam: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$

If you can tune only one thing, tune α

A rough guide from experience, not a law: what matters most depends on the problem.



Try Random Values: Don’t Use a Grid



Question: both searches train 9 models. How many different values of the **important** parameter does each one try?

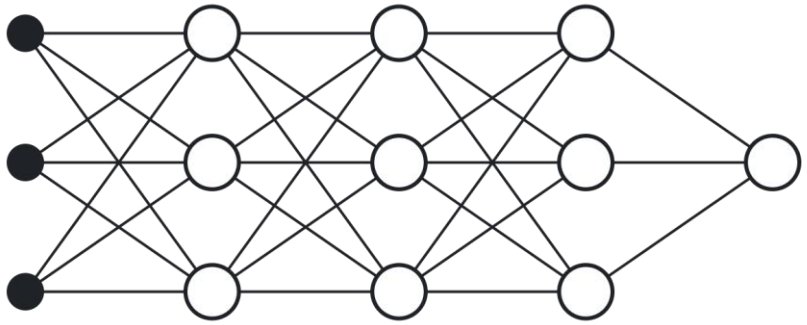
→ Grid: only **3** values. Random: **9** values, so a much better chance to land near the peak of the green curve.

Figure: J. Bergstra and Y. Bengio, Random search for hyper-parameter optimization, JMLR 2012.

8. Batch Normalization: the Idea

Normalizing the **inputs** makes the cost round and speeds up learning (Lecture 3):

$$\mathbf{X} := \frac{\mathbf{X} - \boldsymbol{\mu}}{\boldsymbol{\sigma}}$$



Question: can we also normalize the values **inside** the network, so that $\mathbf{W}^{[3]}, \mathbf{b}^{[3]}$ train faster? Layer 3 sees $\mathbf{A}^{[2]}$ as its input.

In practice we normalize $\mathbf{Z}^{[l]}$, before the activation.

S. Ioffe and C. Szegedy, Batch normalization, ICML 2015.

Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift

Sergey Ioffe
Google Inc., *sioffe@google.com*

Christian Szegedy
Google Inc., *szegedy@google.com*

Internal Covariate Shift

In a deep neural net, the data goes through many transformations as it passes through the layers.

Covariate shift: a change in the distribution of the *input* data.

Internal Covariate Shift: In deep networks,

the distribution of *activations* changes from layer to layer during training.

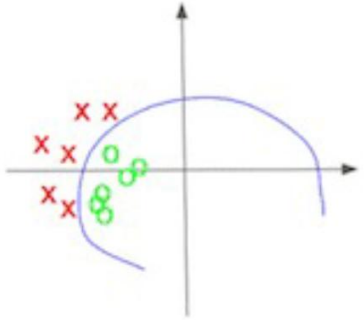
Example of Covariate Shift

Goal is to build a binary classifier for roses.



Rose
(y=1)

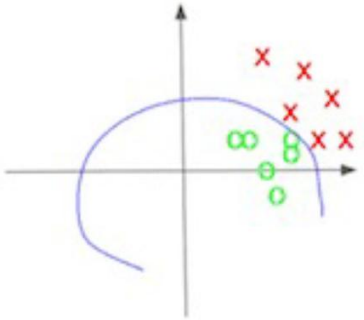
Not Rose
(y=0)





Rose
(y=1)

Not Rose
(y=0)



Internal Covariate Shift in Deep Learning

Covariate Shift: the distribution of the *input* data changes

Internal Covariate Shift: In deep networks,
the distribution of *activations* (the output of each layer)
changes from layer to layer *during training*.

Problem with ICS:
Each layer learns in a different "input space"
as the distributions of activations shift.
This makes training unstable and inefficient.

Consequences of Internal Covariate Shift in Deep Learning

- Slower Training: The model may take longer to converge.
- Unstable Gradients: The gradients can become unstable, leading to vanishing or exploding gradients.
- Difficulty in Hyperparameter Tuning: It becomes more challenging to tune hyperparameters, such as learning rates.
- Reduced Model Performance: The model may achieve worse accuracy and generalization to new data.
- Increased Sensitivity to Initialization: The model becomes more sensitive to the initial weights.

How Batch Normalization Works

Problem: the distribution of *activations* (the output of each layer) changes from layer to layer *during training*.

Batch Normalization:

Normalizes pre-activations within each mini-batch

to have 0 mean and 1 variance.

Done for each feature independently.

Implementing Batch Norm

For one layer l, given $z^{(1)}, \ldots, z^{(m)}$ for the m examples of the mini-batch:

$$\mu = \frac{1}{m} \sum_i z^{(i)}$$

$$\sigma^2 = \frac{1}{m} \sum_i \left(z^{(i)} - \mu\right)^2$$

$$z^{(i)}_{\text{norm}} = \frac{z^{(i)} - \mu}{\sqrt{\sigma^2 + \varepsilon}}$$

$$\tilde{z}^{(i)} = \gamma\, z^{(i)}_{\text{norm}} + \beta$$

Learnable γ and β

γ and β have shape $(1, n^{[l]})$ and are learned by gradient descent, like $\mathbf{W}$. They let the network choose any mean and variance for each unit.

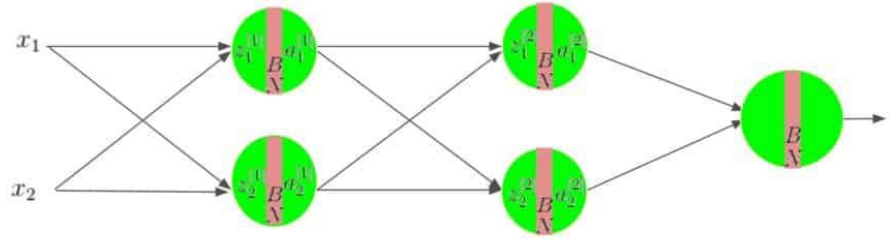
If $\gamma = \sqrt{\sigma^2 + \varepsilon}$ and $\beta = \mu$, then $\tilde{z} = z$: batch norm can even undo itself.

Then use $\tilde{z}^{[l](i)}$ instead of $z^{[l](i)}$. Row convention: μ and σ^2 are computed per unit (column), over the rows of the mini-batch.

Adding Batch Norm to a Network

$$\mathbf{X} \xrightarrow{\mathbf{W}^{[1]}, \mathbf{b}^{[1]}} \mathbf{Z}^{[1]} \xrightarrow[\text{batch norm}]{\beta^{[1]}, \gamma^{[1]}} \tilde{\mathbf{Z}}^{[1]} \rightarrow \mathbf{A}^{[1]} = g^{[1]}(\tilde{\mathbf{Z}}^{[1]}) \xrightarrow{\mathbf{W}^{[2]}, \mathbf{b}^{[2]}} \mathbf{Z}^{[2]} \xrightarrow[\text{batch norm}]{\beta^{[2]}, \gamma^{[2]}} \tilde{\mathbf{Z}}^{[2]} \rightarrow \dots$$

- Parameters: $\mathbf{W}^{[l]}, \mathbf{b}^{[l]}, \beta^{[l]}, \gamma^{[l]}$. But $\mathbf{b}^{[l]}$ is useless: subtracting the mean removes any constant. Drop it (bias=False); $\beta^{[l]}$ plays its role.
- Each mini-batch $\mathbf{X}^{\{t\}}$ is normalized with its **own** μ and σ^2.
- Backprop gives $d\mathbf{W}^{[l]}, d\beta^{[l]}, d\gamma^{[l]}$; update them with momentum, RMSprop or Adam.
- Careful: this $\beta^{[l]}$ has nothing to do with the β of momentum or Adam.



$$z^{[l]} = W^{[l]} a^{[l-1]} \longrightarrow \mu^{[l]} = \frac{1}{m}\sum_i z^{[l](i)}$$
$$\sigma^{[l]2} = \frac{1}{m}\sum_i (z^{[l](i)} - \mu^{[l]})^2$$
$$z^{[l](i)}_{norm} = \frac{z^{[l](i)} - \mu^{[l]}}{\sqrt{\sigma^{[l]2} + \epsilon}}$$
$$\tilde{z}^{[l](i)} = \gamma^{[l]} z^{[l](i)}_{norm} + \beta^{[l]} \longrightarrow a^{[l]} = g^{[l]}(\tilde{z}^{[l]})$$

Putting It Together in PyTorch

```
model = nn.Sequential(
    nn.Linear(784, 256, bias=False), nn.BatchNorm1d(256), nn.ReLU(), nn.Dropout(p=0.2),
    nn.Linear(256, 128, bias=False), nn.BatchNorm1d(128), nn.ReLU(), nn.Dropout(p=0.2),
    nn.Linear(128, 10))                          # logits; softmax is inside the loss
for layer in model:
    if isinstance(layer, nn.Linear):
        nn.init.kaiming_normal_(layer.weight, nonlinearity="relu")      # He

loss_fn = nn.CrossEntropyLoss()
opt = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-4)   # Adam + L2
sched = torch.optim.lr_scheduler.StepLR(opt, step_size=10, gamma=0.5)   # decay
loader = DataLoader(train_ds, batch_size=128, shuffle=True)              # mini-batches
```

Training: model.train(), then for each mini-batch: forward, loss, zero_grad, backward, step; sched.step() once per epoch. Evaluation: epoch. Evaluation: model.eval() and torch.inference_mode().

Debugging Training: Problems, Diagnostics, Workarounds

Problem	Diagnostic	Workaround
Incorrect gradients	Finite differences	Fix them, or use autograd
Local optima	(hard)	Random restarts
Symmetries	Visualize $\mathbf{W}$	Initialize $\mathbf{W}$ randomly
Slow progress	Slow, linear training curve	Increase α, momentum
Instability	Cost increases	Decrease α, momentum
Oscillations	Fluctuations in training curve	Decrease α, momentum
Fluctuations	Fluctuations in training curve	Decay α, iterate averaging
Dead or saturated units	Activation histograms	Initial scale of $\mathbf{W}$, ReLU
Ill-conditioning	(hard)	Adam, batch norm, second-order

Question: for each problem, what would you try?

Figure: A. Gao, CSC413 Neural Networks and Deep Learning, University of Toronto.

Practical Defaults

- **Optimizer:** Adam with a small α (start at 10^{-3}). Try SGD with momentum if you have time.
  - If Adam with a small α fails badly, the bug is almost always somewhere else.
- **Initialization:** PyTorch's defaults, or He for ReLU networks (Lecture 3).
- **Batch size:** the largest power of 2 that fits in memory; re-tune α when you change it.
- **Architecture:** start simple, with a design known to work on a similar task; add batch norm, dropout, … later.
- **More:** Google's Deep Learning Tuning Playbook, github.com/google-research/tuning_playbook

After the CSC413 tutorial "How to train your neural network", University of Toronto.

From the Lecture to PyTorch

In Lectures 3 and 4	In PyTorch
Mini-batches $\mathbf{X}^{\{t\}}$	DataLoader(ds, batch_size=64, shuffle=True)
L2 regularization	weight_decay= in the optimizer
Dropout with keep_prob	nn.Dropout(p=1 - keep_prob)
He initialization	nn.init.kaiming_normal_(w, nonlinearity="relu")
Momentum, Nesterov	optim.SGD(momentum=0.9, nesterov=True)
RMSprop, Adam	optim.RMSprop, optim.Adam, optim.AdamW
Learning rate decay	optim.lr_scheduler.StepLR, ExponentialLR, …
Batch norm	nn.BatchNorm1d(n), with bias=False in the linear layer
Training vs. test behavior	model.train() / model.eval()

Summary

We can now train deep networks fast and reliably!

- **Mini-batch gradient descent:** many cheap, noisy steps per epoch; batch size 32 to 512.
- **The landscape:** saddle points, plateaus and narrow ravines slow gradient descent down, more than bad local minima.
- **Exponentially weighted averages** $v_t = \beta v_{t-1} + (1-\beta)\theta_t$, with bias correction $v_t/(1-\beta^t)$.
- **Momentum** averages $d\mathbf{W}$; **RMSprop** divides by the average of $d\mathbf{W}^2$; **Adam** does both.
- **Learning rate:** the most important hyperparameter; decay it, but not too early.
- **Tuning:** random search, log scale. **Batch norm:** normalize $\mathbf{Z}^{[l]}$ per mini-batch, learnable γ, β.



Thank you!